DRAWING COMPLIANCE CHECKER

A release gate for drawings: concept demo for GRSE GAINS 2026

Prepared 12 October 2026

Code Explanation

How the code is organised, and what each part does

> This document is for someone who will read or change the code. It explains the structure, follows one upload through the files, and shows the key functions. The code excerpts are copied from the source files when the document is built, so they match the code as it is. Read *Project Overview and Flow* first for what the tool does.

> **Everything in this project is mock data.** The drawings, the drawing register, the bills of materials, the work orders and the rules with their clause numbers were invented for the demonstration. None of it comes from GRSE or from a classification society.

Contents

1. Technology choices
2. Map of the repository
3. Architecture in one picture
4. Following one upload through the code
5. Backend: foundations
6. Backend: the checking engine
7. Backend: the API
8. The database
9. API reference
10. Frontend
11. Tests
12. How to change things
13. Known shortcuts

1. Technology choices

Layer	Choice	Why
API	FastAPI on uvicorn, Pydantic for request and response shapes	Typed contracts, and a built-in test page at /docs.
Database	SQLite through SQLAlchemy, Alembic for migrations	One file and nothing to install; tables change without losing data; one setting moves it to PostgreSQL.
PDF	PyMuPDF	Reads text with positions, finds tables, renders pages to images, and draws the mock drawings.
OCR	Tesseract, called as a program	Runs on the machine, gives a confidence per word. No Python OCR library is needed.
Images	NumPy and OpenCV	The pixel overlay, and straightening two scans against each other.
AI summary	Any service speaking the OpenAI chat format (Groq by default), called with httpx	Optional; the provider and model are settings.
Frontend	React with TypeScript, built by Vite	Typed API responses catch mistakes early.
UI components	Ant Design, recoloured in one theme file	Dense tables, tags, forms and layout built in.
Server state	TanStack Query	Fetching, caching and refetching after a change, without hand-written loading flags.
Tests	pytest	The engine and the API are covered; the web app is not yet.

2. Map of the repository

```
GRSE/
  backend/
    app/
      main.py            starts the API; serves the built web app when frontend/dist exists
      config.py          folder locations, limits, optional settings read from the environment or .env
      db.py              database connection, migrations at start-up, backups
      models.py          the database tables
      schemas.py         the shape of what the API accepts and returns
      auth.py            password hashing, sessions, role checks, the audit helper
      rulebook.py        the rule file joined with who approved each rule
      api/
        auth.py          sign in/out, reviewer registration, user management
        checks.py        read a drawing, create a check, images, decision, comments, AI summary
        erp.py           register (and admin edits), drawing detail, release gate, class approvals
        reference.py     rule list, settings, audit log
      core/              the checking logic; knows nothing about the web or the database
        extract.py       PDF -> plain data, for vector PDFs and for scans
        ocr.py           runs Tesseract
        rules.py         the check types, the rule runner, the verdict
        formula.py       works out a rule's formula safely (arithmetic only)
        bom.py           parts list against the ERP bill of materials
        diff.py          what changed between two revisions: change list and overlay image
        impact.py        work orders and purchase orders affected by a change
        gate.py          may this work order be released?
        explain.py       the optional AI summary
    data/rules/rules.json   the mock rules
    migrations/             one file per change to the database tables
    scripts/                make_samples, load_extract, seed_users, approve_demo_rules, new_migration
    tests/                  test_core, test_api, test_db
    var/                    created at run time: database, backups, uploads
  frontend/
    src/
      main.tsx  App.tsx  theme.ts  api.ts  index.css
      components/          layout, tags, tables, sheet viewer, decision panel, forms
      pages/               one file per screen
  demo/                     start script, logins, mock ERP files, mock drawings, video
  docs/                     these documents and the script that builds them
  learning.md  README.md
```

3. Architecture in one picture

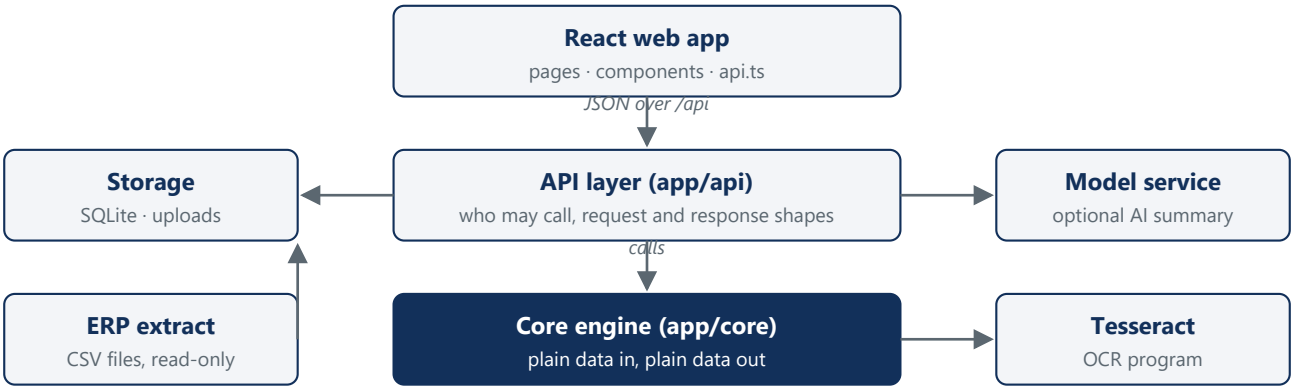


Figure 1. The layers. The API layer is the only part that talks to both the database and the engine.

> **The rule that keeps this tidy:** nothing in `app/core` imports the web framework or the database. Each function there takes plain Python data (dictionaries and lists) and returns plain data. That is why the engine can be tested without starting a server, and could be reused from a command line or another framework.

4. Following one upload through the code

1	**The browser sends the file** `pages/NewCheck.tsx` posts the PDF to `POST /api/checks/read` to show what was read, then to `POST /api/checks` to run the check. For a scan it also sends the confirmed title-block values.
2	**The request is checked** `api/checks.py: create_check`. The `submitter` dependency from `auth.py` lets through only an engineer or an admin. `read_drawing` refuses files over 25 MB or that do not start with the PDF signature.
3	**The drawing is read** `core/extract.py: extract` returns one dictionary: `fields`, `parts`, `standards`, `lines`, `text`, `source`. A page with no text goes down the OCR path in the same file.
4	**A scan is confirmed** `apply_confirmation` replaces what OCR read with what the person confirmed and remembers the original as `read_as`.
5	**The ERP's view is gathered** The register row, the bill of materials for this drawing revision and the particulars of its hull are read from the database and passed to the engine as one bundle called `erp`.
6	**The rules run** `rulebook.py: rulebook` gives the rules with their approval state; `core/rules.py: run_checks` calls the check function named by each approved rule and collects findings; `verdict` reduces them to one word.
7	**The previous revision is compared** `core/diff.py: compare` produces the change list; `core/impact.py: impact` turns it into affected work orders and purchase orders.
8	**Everything is stored** One `Check` row with its `Finding` rows, the uploaded file under a generated name, and snapshots (register row, BOM comparison, revision changes, impact) as JSON. `audit(...)` adds a line to the audit log.
9	**The report is shown** The response has the shape `schemas.CheckOut`. `pages/CheckReport.tsx` draws it; images come from `GET /api/checks/{id}/sheet.png` and `diff.png`, rendered on request.

5. Backend: foundations

config.py: where things are

Paths and limits in one place. Read-only inputs live in `demo/` and `backend/data/rules/`; everything the tool writes lives in `backend/var/`. Optional settings come from environment variables or a `.env` file: `GRSE_DATABASE_URL`, `GRSE_EXPLAINER_KEY / _URL / _MODEL`, `GRSE_TESSERACT`, `GRSE_COOKIE_SECURE`, `GRSE_VAR_DIR`.

db.py: the database, and upgrading it safely

`migrate()` runs every time the server starts. It finds which migration the database is on, takes a backup if newer ones exist, then applies them. A database from before migrations existed is recognised and marked as being on the first migration.

From `backend/app/db.py`:

```
def migrate(target=engine) -> None:
    """Bring the database up to the tables the code expects, keeping every row that is in it.

    Runs at every start. With nothing to change it does nothing; before changing anything it takes a
           backup.
    """
    cfg = Config()
    cfg.set_main_option("script_location", str(config.BACKEND_DIR / "migrations"))
    cfg.attributes["engine"] = target
    head = ScriptDirectory.from_config(cfg).get_current_head()

    with target.connect() as connection:
        has_tables = bool(inspect(connection).get_table_names())
        current = MigrationContext.configure(connection).get_current_revision()

    if has_tables and current is None:
        # A database created before migrations existed: its tables are exactly what the baseline
                creates.
        command.stamp(cfg, BASELINE)
        current = BASELINE
    if has_tables and current != head:
        backup(f"before-{head}", target)
    command.upgrade(cfg, "head")
```

models.py: the tables

SQLAlchemy classes, one per table. The first group is a copy of the ERP extract; the second is what the tool itself records. Section 8 lists them. Two helpers live here: `UTCDateTime` puts the timezone back on times read from SQLite, and `as_dict` turns a row into plain data for the engine.

auth.py: who is signed in, and what they may do

- Passwords are stored only as a salted scrypt hash.
- A session is a random token in an HttpOnly cookie; the database stores a hash of the token with an 8-hour expiry.
- `current_user` is a FastAPI dependency: add it to a route and the route requires a signed-in, active user.
- `require(...)` builds a dependency that lets through certain roles. An admin passes everywhere.

From `backend/app/auth.py`:

```
def require(*roles: str):
    """A dependency that lets through only the given roles. An admin is let through everywhere."""

    def allowed(user: User = Depends(current_user)) -> User:
        if user.role != "admin" and user.role not in roles:
            raise HTTPException(403, "Your role does not allow this.")
        return user

    return allowed
```

6. Backend: the checking engine

extract.py: from PDF to plain data

The reading is **template-based**: a few constants at the top of the file describe the mock sheet, namely where the title block sits, the labels printed in it, and how wide the value under each label is. To support a real title block, these constants are what changes.

From `backend/app/core/extract.py`:

```
def _title_block(page, clip) -> dict:
    words = page.get_text("words", clip=clip)
    fields = {}
    for name, (label, width) in FIELDS.items():
        value, bbox = "", None
        hit = next((word for word in words if word[4] == label), None)
        if hit:
            box = pymupdf.Rect(hit[0] - 1, hit[3], hit[0] + width, hit[3] + VALUE_HEIGHT)
            value = " ".join(page.get_textbox(box).split())
            bbox = list(box) if value else None
        fields[name] = {"value": value, "bbox": bbox}
    return fields
```

For a vector PDF the parts list comes from PyMuPDF's table finder. For a scan, `_parts_scan` rebuilds it from word positions: the heading row gives the column positions, the words beneath give the rows, and an empty cell is read again on its own by `ocr.read_cell`. Both paths return the same dictionary, so the rest of the code does not care which kind of file it was.

ocr.py: calling Tesseract

Tesseract is run as a separate program: the page image goes in on standard input and a table of words comes back. Two passes with different page settings are merged.

From `backend/app/core/ocr.py`:

```
def read_words(png: bytes, dpi: int) -> list:
    """Words on the picture: {"text", "confidence", "box" in PDF points, "line" (words sharing it are
            one line of text)}."""
    tesseract = find_tesseract()
    if tesseract is None:
        raise RuntimeError("Tesseract is not installed")
    words = _run(tesseract, png, dpi, PASSES[0])
    for extra in _run(tesseract, png, dpi, PASSES[1]):
        # Keep a word from the second pass only where the first pass found nothing.
        if not any(_inside(_centre(extra["box"]), word["box"], margin=1) for word in words):
            words.append(extra)
    return words
```

rules.py: rules are data, checks are code

Every check type is a small function with the same signature: it receives the drawing, the ERP bundle and the rule's parameters, and *yields* one (evidence, position) pair for each problem. This one checks plate thickness in the title block and on every parts-list row:

From backend/app/core/rules.py:

```
def min_thickness(d, erp, p):
    for where, raw, bbox in _everywhere(d, "thickness_mm"):
        try:
            thickness = float(raw)
        except ValueError:
            yield f"{where}: thickness '{raw}' could not be read as a number", bbox
            continue
        if thickness < p["min_mm"]:
            yield f"{where}: thickness {thickness:g} mm is below the minimum {p['min_mm']:g} mm", bbox
```

The rule that uses it is an entry in backend/data/rules/rules.json:

```
{
  "id": "CHK-04",
  "name": "Plate thickness meets the absolute minimum",
  "type": "min_thickness",
  "severity": "critical",
  "clause": "MOCK-CLS 3.4.2",
  "standard": "MOCK-CLS STEEL SHIP RULES",
  "edition": 2024,
  "params": { "min_mm": 6 },
  "demo_state": "approved"
}
```

The file *defines* a rule; it does not approve it. (demo_state only tells the demo's setup script which rules to approve the first time.)

The runner ties the two together, and the verdict is three lines:

From backend/app/core/rules.py:

```
def run_checks(drawing: dict, erp: dict, rules: list) -> list:
    findings = []
    for rule in rules:
        if rule["state"] != "approved":
            continue
        for evidence, bbox in CHECKS[rule["type"]](drawing, erp, rule.get("params", {})):
            findings.append({
                "rule_id": rule["id"],
                "rule_name": rule["name"],
                "severity": rule["severity"],
                "clause": rule["clause"],
                "standard": f"{rule['standard']}, edition {rule['edition']}",
                "evidence": evidence,
                "bbox": bbox,
            })
    # Worst first; within a severity, rule order is kept.
    return sorted(findings, key=lambda f: SEVERITIES.index(f["severity"]))
```

From backend/app/core/rules.py:

```
def verdict(findings: list) -> str:
    severities = {f["severity"] for f in findings}
    return "BLOCKED" if "critical" in severities else "NEEDS_REVIEW" if "major" in severities else
            "PASS"
```

Rules by zone and by formula

A rule can carry one formula per structural zone. The check reads the zone and stiffener spacing from the title block and the ship's length from the ERP bundle, works out the requirement, and puts the working into the evidence:

From backend/app/core/rules.py:

```
def required_thickness(d, erp, p):
    """Thickness worked out from a formula that depends on the zone, the ship's length L (m) and the
           stiffener spacing s (mm)."""
    zone = _val(d, "zone")
    formula = p["formulas"].get(zone)
    if formula is None:
        return  # this rule sets no requirement for this zone
    ship, spacing = erp["ship"], _number(_val(d, "spacing_mm"))
    if ship is None:
        yield (f"Hull '{_val(d, 'hull_no') or '(unreadable)'}' is not in the ERP ship particulars, "
               f"so the thickness required in zone {zone} could not be calculated"), _box(d, "hull_no")
        return
    if spacing is None:
        yield f"Stiffener spacing could not be read, so the thickness required in zone {zone} could not
                be calculated", _box(d, "spacing_mm")
        return
    length = ship["rule_length_m"]
    required = evaluate(formula, {"L": length, "s": spacing})
    for where, raw, _, bbox in _members(d, p["member"]):
        thickness = _number(raw)
        if thickness is not None and thickness < required:
            yield (f"{where}: {thickness:g} mm is below the {required:.1f} mm required in zone {zone} "
                   f"({formula}, with L = {length:g} m and s = {spacing:g} mm)"), bbox
```

The formula is text in the rule file, so it must never be run as code. `core/formula.py` parses it and allows only numbers, named quantities, arithmetic, brackets and three functions. Anything else is refused, and a rule with a broken formula is rejected when the rule file is read.

From backend/app/core/formula.py:

```
def evaluate(formula: str, values: dict) -> float:
    """values: the named quantities the formula may use, e.g. {"L": 142.0, "s": 700.0}."""

    def work_out(node):
        if isinstance(node, ast.Constant) and isinstance(node.value, (int, float)) and not
                isinstance(node.value, bool):
            return node.value
        if isinstance(node, ast.Name):
            if node.id not in values:
                raise ValueError(f"'{node.id}' is not a quantity this formula can use ({',
                        '.join(sorted(values))})")
            return values[node.id]
        if isinstance(node, ast.BinOp) and type(node.op) in OPERATORS:
            return OPERATORS[type(node.op)](work_out(node.left), work_out(node.right))
        if isinstance(node, ast.UnaryOp) and type(node.op) in OPERATORS:
            return OPERATORS[type(node.op)](work_out(node.operand))
        if isinstance(node, ast.Call) and isinstance(node.func, ast.Name) and node.func.id in FUNCTIONS
                and not node.keywords:
            return FUNCTIONS[node.func.id](*(work_out(argument) for argument in node.args))
        raise ValueError("only numbers, named quantities, + - * / **, brackets, sqrt, min and max are
                allowed")

    try:
        return float(work_out(ast.parse(formula, mode="eval").body))
    except (SyntaxError, ZeroDivisionError, OverflowError, TypeError) as e:
        raise ValueError(f"the formula '{formula}' cannot be worked out: {e}") from e
```

rulebook.py: which rules are in force

An approval is a row in the database holding a fingerprint (a hash) of the rule's content. A rule is in force only if its newest approval has not been withdrawn and its fingerprint still matches the rule file. Editing a rule therefore takes it out of use until someone approves the new content.

From backend/app/rulebook.py:

```
def rulebook(session: Session) -> list:
    """Every rule with its state: "approved" (runs), "draft" (never approved) or "changed" (edited since
            it was approved)."""
    latest = {}
    for approval in session.scalars(select(RuleApproval).order_by(RuleApproval.id)):
        latest[approval.rule_id] = approval
    rules = []
    for rule in load_rules(config.RULES_FILE):
        approval, fingerprint = latest.get(rule["id"]), content_hash(rule)
        in_force = approval is not None and approval.withdrawn_at is None
        state = "approved" if in_force and approval.content_hash == fingerprint else "changed" if
                in_force else "draft"
        rules.append({k: v for k, v in rule.items() if k != "demo_state"} | {
            "state": state,
            "content_hash": fingerprint,
            "approved_by": approval.approved_by if in_force else None,
            "approved_at": approval.approved_at.isoformat() if in_force else None,
            "approval_note": approval.note if in_force else None,
        })
    return rules
```

bom.py: parts list against the bill of materials

From backend/app/core/bom.py:

```
def compare_bom(parts: list, bom: list) -> list:
    """One row per part number found on either side. result: match, mismatch, not_in_erp or
            not_on_drawing."""
    in_erp = {item["part_no"]: item for item in bom}
    rows = []
    for part in parts:
        item = in_erp.pop(part["part_no"], None)
        drawing = {column: _text(part[column]) for column in COLUMNS}
        erp = item and {column: _text(item[column]) for column in COLUMNS}
        differences = [
            {"column": label, "drawing": drawing[column], "erp": erp[column]}
            for column, label in COLUMNS.items() if erp and drawing[column] != erp[column]
        ]
        rows.append({
            "part_no": part["part_no"], "description": part["description"], "drawing": drawing, "erp":
                    erp,
            "result": "not_in_erp" if erp is None else "mismatch" if differences else "match",
            "differences": differences, "bbox": part["bbox"],
        })
    for item in in_erp.values():
        rows.append({
            "part_no": item["part_no"], "description": item["description"], "drawing": None,
            "erp": {column: _text(item[column]) for column in COLUMNS},
            "result": "not_on_drawing", "differences": [], "bbox": None,
        })
    return rows
```

diff.py: what changed between two revisions

`compare(old, new)` returns a flat list of changes, each with an area (Title block, Parts list, Sheet text), a kind (added, removed, changed), the old and new values and their positions on each sheet. Other text is matched *by wording, not position*, so notes that merely moved are not reported. `overlay_png` renders both sheets, straightens scans with OpenCV, and colours each pixel grey, red or green, with a tolerance so that tiny rendering differences are not shown as changes.

From backend/app/core/diff.py:

```
def _only_in(lines, other) -> list:
    """Lines with no partner of the same text in `other`. Repeated text is matched one for one."""
    available = Counter(key(line["text"]) for line in other)  # letters and digits only: spacing and
            punctuation are not changes
    unmatched = []
    for line in lines:
        if available[key(line["text"])]:
            available[key(line["text"])] -= 1
        else:
            unmatched.append(line)
    return unmatched
```

impact.py and gate.py: the ERP questions

From backend/app/core/impact.py:

```
def impact(changes: list, from_revision: str, work_orders: list, purchase_orders: list) -> dict:
    """changes: the list from diff.compare(). work_orders: those for this drawing. purchase_orders: all
            of them."""
    affected = {}  # part number -> what happened to it
    for change in changes:
        if change["area"] != "Parts list":
            continue
        if change["kind"] == "changed":
            label = change["what"].removeprefix(change["part_no"]).strip()
            text = f"{label} {change['old']} to {change['new']}"
        else:
            text = change["kind"]
        affected.setdefault(change["part_no"], []).append(text)

    return {
        "from_revision": from_revision,
        "work_orders": [order for order in work_orders if order["rev"] == from_revision],
        "purchase_orders": [
            {**po, "change": "; ".join(affected[po["part_no"]])}
            for po in purchase_orders if po["state"] == "OPEN" and po["part_no"] in affected
        ],
    }
```

The gate takes four plain facts and returns a status with reasons. Because it is a pure function, every combination is covered by a table of test cases.

From `backend/app/core/gate.py`:

```
def gate(order: dict, drawing: dict | None, check: dict | None, approval: dict | None) -> dict:
    """order: the work order. drawing: its register row. check: the latest check of that drawing
            revision.
    approval: the class or customer approval of that revision. Any of the last three may be missing."""
    reasons = []  # (level, text)

    if drawing is None:
        reasons.append(("BLOCKED", "The drawing is not in the register"))
    else:
        if order["rev"] != drawing["current_rev"]:
            reasons.append(("BLOCKED", f"Revision {order['rev']} is superseded; the register's current
                    revision is {drawing['current_rev']}"))
        if drawing["status"] not in RELEASABLE:
            reasons.append(("BLOCKED", f"Register status is '{drawing['status']}', not approved for
                    production"))

    if check is None:
        reasons.append(("WARN", "This revision has not been checked"))
    elif check["decision"] is None:
        level = "BLOCKED" if check["verdict"] == "BLOCKED" else "WARN"
        reasons.append((level, f"Check #{check['id']} is {check['verdict'].replace('_', ' ')} and awaits
                 the admin's decision"))
    elif check["decision"]["decision"] == "REJECTED":
        reasons.append(("BLOCKED", f"Rejected by {check['decision']['decided_by']} (check
                #{check['id']})"))

    if approval is None:
        reasons.append(("WARN", "No class or customer approval is on record for this revision"))
    elif approval["state"] != "APPROVED":
        reasons.append(("WARN", f"{approval['submitted_to']} approval: {approval['state']}"))

    status = max((level for level, _ in reasons), key=LEVELS.index, default="CLEAR")
    return {"status": status, "reasons": [text for _, text in reasons]}
```

explain.py: the optional AI summary

`as_text` builds the exact text that is sent (the change list and nothing else). `explain` posts it with a fixed instruction, and reports errors by status code only so the key can never reach a screen. Then every number in the answer is checked against what was sent:

From `backend/app/core/explain.py`:

```
def unsupported_numbers(explanation: str, source: str, count: int) -> list:
    """Numbers in the explanation that are nowhere in the change list: a sign the model made something
            up."""
    known = set(NUMBER.findall(source)) | {str(count)}
    return sorted({n for n in NUMBER.findall(explanation) if n not in known})
```

7. Backend: the API

Each file in `app/api` is a FastAPI router. A route's job is to check who is calling, validate the input, fetch what the engine needs, call the engine, store the result and write the audit line. The logic itself stays in `app/core`.

The reviewer's decision shows the pattern, including the three safeguards (decide once, not your own submission, a reason when going against the verdict):

From backend/app/api/checks.py:

```
def decide(check_id: int, body: DecisionIn, user: User = Depends(admin), session: Session =
        Depends(get_session)):
    check = get_or_404(session, check_id)
    if check.decision is not None:
        raise HTTPException(409, "This check has already been decided.")
    own = check.submitted_by == user.username
    reason = body.reason.strip()
    outcome = (check.verdict, body.decision)
    if outcome not in AGREES and not reason:
        raise HTTPException(422, "A reason is required when the decision does not simply follow the
                verdict.")

    check.decision = Decision(decision=body.decision, is_override=outcome in OVERRIDES, reason=reason,
            decided_by=user.username)
    audit(session, user.username, user.role, "decision.override" if outcome in OVERRIDES else
            "decision.recorded",
          f"check {check.id}", f"{check.drawing_no} rev {check.revision}: verdict {check.verdict},
                  decision {body.decision}. {reason}"
          + (" (decided by the admin who submitted it)" if own else ""))
    session.commit()
    return check
```

The release gate route gathers the facts for every work order and hands each set to the pure `gate` function:

From backend/app/api/erp.py:

```
def gate_rows(session: Session, orders) -> list:
    drawings = {d.drawing_no: as_dict(d) for d in session.scalars(select(Drawing))}
    approvals = approvals_by_revision(session)
    # shortcut: reads every check to find the latest per drawing revision; query per work order when
            there are thousands
    latest = {}
    for check in session.scalars(select(Check).order_by(Check.id)):
        latest[(check.drawing_no, check.revision)] = {
            "id": check.id, "verdict": check.verdict, "decision": check.decision and
                    as_dict(check.decision),
        }
    rows = []
    for order in orders:
        key = (order.drawing_no, order.rev)
        drawing, check = drawings.get(order.drawing_no), latest.get(key)
        rows.append({
            **as_dict(order),
            "current_rev": drawing and drawing["current_rev"],
            "check_id": check and check["id"],
            **gate(as_dict(order), drawing, check, approvals.get(key)),
        })
    return rows
```

8. The database

Table	Holds	Filled by
`drawings`	Number, title, current revision, status, source (erp or manual)	ERP extract; admin edits
`drawing_revisions`	Every revision, its status and issue date	ERP extract; admin edits
`change_notes`	Engineering change notes: what, when, why	ERP extract
`bom_items`	Parts per drawing revision: material, thickness, quantity	ERP extract
`work_orders`	Order, drawing, revision released against, state	ERP extract
`purchase_orders`	Order, part, quantity, state	ERP extract
`class_approvals`	Submitted to whom, when, outcome	ERP extract
`ship_particulars`	Hull number and the ship's rule length, for calculated rules	ERP extract
`rule_approvals`	Who approved which rule, a fingerprint of its content, and any withdrawal	Tool
`users`	Username, name, role, password hash, status	Logins file; registration
`auth_sessions`	Hash of a session token, user, expiry	Sign-in
`checks`	One run: drawing, verdict, what was read, and snapshots as JSON (register row, revision changes, BOM comparison, impact, AI summary)	Tool
`findings`	Each finding, with a copy of the rule's name, clause and standard	Tool
`decisions`	The reviewer's decision, reason, override flag (one per check)	Tool
`comments`	Comments on a check	Tool
`audit_log`	Who did what and when	Tool

Why snapshots and copies? A report must stay true after the ERP data or a rule changes. So a check keeps the register row as it was, and a finding keeps its own copy of the clause it was judged against, instead of pointing at data that may later change.

Migrations. `backend/migrations/versions/` has one numbered file per change. After editing `models.py`, run `python -m scripts.new_migration 0005 "what changed"` from `backend/`, read the generated file, and keep it. `tests/test_db.py` fails if the tables the migrations build differ from the tables the code expects.

9. API reference

Route	Who	Does
`POST /api/auth/login`	Anyone	Sign in; sets the session cookie
`POST /api/auth/logout`	Signed in	End the session
`GET /api/auth/me`	Signed in	Who is signed in
`POST /api/auth/register`	Anyone	Ask for an engineer or reviewer account (pending until approved)
`GET /api/users`	Admin	All accounts
`POST /api/users/{u}/status`	Admin	Approve, disable or re-enable
`POST /api/checks/read`	Signed in	What the tool reads from a PDF; stores nothing
`POST /api/checks`	Signed in	Run and store a check
`GET /api/checks`	Signed in	All checks, newest first
`GET /api/checks/{id}`	Signed in	One report
`GET /api/checks/{id}/sheet.png`	Signed in	The sheet as an image (also previous.png, diff.png)
`POST /api/checks/{id}/decision`	Admin	Approve or reject
`POST /api/checks/{id}/comments`	Signed in	Add a comment
`POST /api/checks/{id}/explanation`	Signed in	Ask for the AI summary (made once)
`GET /api/register`	Signed in	The register
`POST /api/register`	Admin	Add a drawing
`GET /api/register/{no}`	Signed in	One drawing with history, BOM, orders, approvals
`PUT /api/register/{no}`	Admin	Update a drawing
`GET /api/gate`	Signed in	Gate status of every work order
`GET /api/gate/{order}`	Signed in	Gate status of one work order
`GET /api/class-approvals`	Signed in	Approvals, longest wait first
`GET /api/rules`	Signed in	The rules with their approval state
`POST /api/rules/{id}/approval`	Admin	Approve a rule as it stands
`POST /api/rules/{id}/withdrawal`	Admin	Take a rule out of use
`GET /api/settings`	Signed in	Whether the AI summary is on, and its model
`GET /api/audit`	Reviewer, admin	The audit log

With the server running, `/docs` shows the same list as an interactive page generated by FastAPI.

10. Frontend

File	Role
`main.tsx`	Starts the app and wraps it in the theme, the data cache and the router. A 401 from any request sends the user back to sign-in.
`App.tsx`	Shows the sign-in page or the app, and lists the routes.
`theme.ts`	Colours in one place: the navy, and the status colours used by tags, banners and sheet markers.
`api.ts`	The types of everything the server returns, the `api()` fetch helper, one hook per kind of data (`useChecks`, `useGate` ...), and `can`, which says what each role may see.
`components/`	Reusable parts: `AppLayout`, `Tags`, `ChecksTable`, `GateTable`, `SheetViewer`, `RevisionChanges`, `ErpChecks`, `DecisionPanel`, `Comments`, `Explanation`, `DrawingForm`.
`pages/`	One file per screen. A page asks for data with a hook and arranges components.
`index.css`	The few styles Ant Design does not cover: the demo banner, sheet markers, print layout.

Three ideas worth understanding

- **Server state lives in a cache.** A page calls a hook such as `useCheck(id)`; TanStack Query fetches, remembers and shares the result. After a change (a decision, an upload, a register edit) the code *invalidates* the affected keys and the screens that use them refetch.
- **The server is the authority on permissions.** `can.submit(user)` only decides what to show. The same rule is enforced again by the API, so hiding a button is never the only protection.
- **Markers on a sheet are percentages.** The server stores a finding's position in PDF points. `SheetViewer` divides by the page size and positions a box by percentage over the image, so markers stay in place at any zoom.

During development two servers run: Vite on port 5173 forwards `/api` to uvicorn on port 8000. For a real run, `npm run build` produces `frontend/dist` and the backend serves it, so there is one process.

11. Tests

File	What it proves
`tests/test_core.py`	The engine: what is read from each sample; the verdict and rule set for each; per-part findings; the BOM comparison; the gate's decision table; change impact; the change list and the overlay; scans read like their originals; crooked scans are straightened; zone and formula rules, and that a formula can contain arithmetic and nothing else.
`tests/test_api.py`	The API: everything needs a sign-in; each role's limits; uploads and bad uploads; decisions and their safeguards; registration and approval; comments; the gate following a decision; scan confirmation; the AI summary (with a stand-in model) and what it sends; admin register edits surviving an extract reload; a rule running only while an approval of its exact content stands.
`tests/test_db.py`	Migrations build exactly the tables the code expects; upgrading keeps the data and takes a backup.

Run them from `backend/` with `.\.venv\Scripts\python.exe -m pytest -q`. Tests use a temporary database and temporary sample drawings, never the real data. The OCR tests are skipped on a machine without Tesseract. The suite never calls the real model service.

12. How to change things

To do this	Change this
Add a rule of an existing kind	Add an entry to `backend/data/rules/rules.json` with a clause, severity and parameters, then approve it as admin on the Rule Library screen.
Change a rule's limit or formula	Edit its entry in the rule file. It shows as “changed” and stops running until an admin approves it again.
Add a new kind of check	Write a function in `core/rules.py` with the `(d, erp, p)` signature that yields (evidence, position); add it to `CHECKS`; add a test.
Support a real title block	Change `TITLE_BLOCK`, `FIELDS` and the parts-list headings at the top of `core/extract.py`; regenerate or add samples; update the tests' expected values.
Add a column or a table	Edit `models.py`, run `scripts.new_migration`, read the generated migration, run the tests.
Give rules more ERP data	Add it to the `erp` bundle built in `api/checks.py: create_check`; every check function already receives the bundle.
Add a screen	A file in `pages/`, a route in `App.tsx`, a menu entry in `components/AppLayout.tsx`, and a hook in `api.ts` if it needs new data.
Change the AI model or provider	Set `GRSE_EXPLAINER_MODEL` / `GRSE_EXPLAINER_URL` in `backend/.env`. No code change.
Move to PostgreSQL	Set `GRSE_DATABASE_URL` and install a PostgreSQL driver. The migrations run there too.
Re-record the video, rebuild these documents	`python demo/record_demo.py` (with the backend environment) and `python docs/build_docs.py`.

13. Known shortcuts

Where a simpler solution with a known limit was chosen on purpose, the code says so in a comment beginning `shortcut:`. This list is collected from the source when the document is built.

Where	The limit, and when to upgrade
`app/api/auth.py:19`	no lockout after repeated failures (each one is in the audit log); add one before real users sign in
`app/api/checks.py:129`	revisions compared as text, as in the register check
`app/api/erp.py:24`	reads every check to find the latest per drawing revision; query per work order when there are thousands
`app/api/reference.py:60`	newest 500 entries, filtered in the browser; add paging when the log outgrows that
`app/core/extract.py:70`	sheet 1 only; loop over pages when multi-sheet drawings are in scope
`app/core/rules.py:42`	revisions compared as text (A < B < C); needs a real ordering if revisions look like P01 or A1
`app/db.py:35`	other databases are backed up with their own tools (pg_dump); wire that in when one is used
`app/models.py:30`	dates are kept as ISO text (YYYY-MM-DD), which sorts correctly;

Larger gaps that are not single lines of code (real title-block layout, real rules, directory sign-in, frontend tests) are listed at the end of the *Project Overview and Flow* document.